

# 1. What is a Program?

A **program** is a set of instructions written in a programming language that tells a computer how to perform a specific task. It takes input, processes it, and produces output.

# 2. What is Programming?

Programming is the process of designing, writing, testing, and maintaining code to create software applications.

## Key Steps in Programming:

1. Problem understanding
2. Algorithm design
3. Coding
4. Testing
5. Debugging
6. Maintenance

# 3. Types of Programming Languages

## Difference between High-level and Low-level Languages:

| High-Level Language   | Low-Level Language              |
|-----------------------|---------------------------------|
| Easy to understand    | Difficult to understand         |
| Human-readable        | Machine-readable                |
| Portable              | Machine dependent               |
| Example: Python, Java | Example: Assembly, Machine code |

## 4. World Wide Web & Internet

### Role of Client and Server:

- **Client:** Requests data (browser, app)
- **Server:** Provides data (web server)

## 5. Network Layers (TCP/IP Model)

The **TCP/IP model** defines how data is transmitted over the internet.

### Layers:

1. Application Layer
2. Transport Layer
3. Internet Layer
4. Network Access Layer

## 6. Client-Server Communication

Client sends a request → Server processes it → Server sends response back.

## 7. Types of Internet Connections

### Broadband vs Fiber:

- **Broadband:** Uses copper cables, slower
- **Fiber-optic:** Uses light signals, faster and more reliable

## 8. Protocols

### HTTP vs HTTPS:

- **HTTP:** Not secure, data in plain text
- **HTTPS:** Secure, uses encryption

## 9. Application Security

### Role of Encryption:

Encryption converts data into a secure format to prevent unauthorized access.

## 10. Software Applications

### System vs Application Software:

- **System Software:** Controls hardware (OS)
- **Application Software:** Performs user tasks (apps)

## 11. Software Architecture

### Importance of Modularity:

Modularity divides software into small parts, making it easy to manage, update, and debug.

## 12. Layers in Software Architecture

Layers separate responsibilities, improving organization and maintainability.

## 13. Software Environments

A **development environment** helps programmers write, test, and debug code efficiently.

## 14. Source Code

- **Source Code:** Human-readable code
- **Machine Code:** Binary code executed by computer

## 15. GitHub

### Importance of Version Control:

Tracks changes, allows collaboration, and prevents data loss.

## 16. GitHub for Students

- Easy collaboration
- Code backup
- Portfolio creation
- Learning teamwork

## 17. Types of Software

### Open-source vs Proprietary:

- **Open-source:** Free, editable
- **Proprietary:** Paid, restricted

## **18. GIT**

### **Role in Collaboration:**

- Multiple developers can work together
- Tracks changes
- Supports branching and merging

## **19. Application Software**

Helps users perform tasks like document creation, communication, and business operations.

## **20. Software Development Process**

### **Stages:**

1. Requirement gathering
2. Design
3. Development
4. Testing
5. Deployment
6. Maintenance

## **21. Software Requirement**

Requirement analysis ensures correct understanding of user needs before development.

## **22. Software Analysis**

Helps identify system functionality and requirements in detail.

## 23. System Design

### Key Elements:

- Architecture
- Data design
- Interface design
- Security

## 24. Software Testing

Testing ensures software is error-free and works correctly.

## 25. Maintenance

### Types:

- Corrective
- Adaptive
- Perfective
- Preventive

## 26. Development (Web vs Desktop)

- **Web Apps:** Run in browser
- **Desktop Apps:** Installed on system

## 27. Web Application

### Advantages:

- No installation
- Accessible anywhere
- Easy updates

## 28. Designing (UI/UX)

UI/UX improves user experience, usability, and satisfaction.

## 29. Mobile Application

### Native vs Hybrid:

- **Native:** Platform-specific, faster
- **Hybrid:** Cross-platform, slower

## 30. DFD (Data Flow Diagram)

DFD shows how data moves within a system.

## 31. Desktop Application

### Pros:

- Fast, works offline

### Cons:

- Requires installation, platform dependent

## **32. Flow Chart**

Flowcharts visually represent logic, making programs easier to understand and design.